

AI Citations

Tool: Gemini

Prompt: pandas extract first 10 chars from string col. and convert to datetime

Answer:

```
# Assuming 'df' is your dataframe and 'col' is the string column
```

```
df['date_cleaned'] = df['col'].astype(str).str[:10]
```

```
df['date'] = pd.to_datetime(df['date_cleaned'])
```

```
# For the other dataframe
```

```
df2['date'] = pd.to_datetime(df2['time_column'])
```

Prompt: bu kısım hyphotesis testlerim import pandas as pd

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
from scipy import stats
```

```
def run_part2_analysis():
```

```
    # 1. Load the merged data created in Part 1
```

```
    try:
```

```
        df = pd.read_csv('istanbul_weather_traffic_merged.csv')
```

```
    except FileNotFoundError:
```

```
        print("Error: Please run Part 1 first to generate 'istanbul_weather_traffic_merged.csv'")
```

```
    return
```

```
    # 2. Rename columns for cleaner analysis and plotting
```

```
    df.rename(columns={
```

```
        'precipitation_sum (mm)': 'Precipitation',
```

```
        'rain_sum (mm)': 'Rain',
```

```
        'snowfall_sum (cm)': 'Snow',
```

```
        'temperature_2m_mean (°C)': 'Temperature',
```

```
        'wind_speed_10m_max (km/h)': 'Wind',
```

```

    'average_traffic_index': 'Traffic'
}, inplace=True)

# Add 'DayType' for Weekend vs Weekday analysis
df['date'] = pd.to_datetime(df['date'])
df['DayType'] = df['date'].dt.dayofweek.apply(lambda x: 'Weekend' if x >= 5 else 'Weekday')

# Hypothesis tests
print("="*50)
print("      HYPOTHESIS TESTING RESULTS      ")
print("="*50)

# Test 1: Precipitation (Wet vs Dry Days)
wet = df[df['Precipitation'] > 0]['Traffic']
dry = df[df['Precipitation'] == 0]['Traffic']
t_stat, p_val = stats.ttest_ind(wet, dry, equal_var=False)
print(f"[1] Precipitation (Wet vs Dry)")
print(f"  Mean Traffic (Wet): {wet.mean():.2f}")
print(f"  Mean Traffic (Dry): {dry.mean():.2f}")
print(f"  P-Value: {p_val:.10f} -> {'Significant' if p_val < 0.05 else 'Not Significant'}")
print("-" * 50)

# Test 2: Rain (Rainy vs No Rain)
rainy = df[df['Rain'] > 0]['Traffic']
no_rain = df[df['Rain'] == 0]['Traffic']
t_stat, p_val = stats.ttest_ind(rainy, no_rain, equal_var=False)
print(f"[2] Rain (Rain vs No Rain)")
print(f"  P-Value: {p_val:.10f} -> {'Significant' if p_val < 0.05 else 'Not Significant'}")
print("-" * 50)

# Test 3: Snow (Snowy vs No Snow)

```

```

snowy = df[df['Snow'] > 0]['Traffic']
no_snow = df[df['Snow'] == 0]['Traffic']
if len(snowy) > 1:
    t_stat, p_val = stats.ttest_ind(snowy, no_snow, equal_var=False)
    print(f"[3] Snow (Snow vs No Snow)")
    print(f"    Mean Traffic (Snowy): {snowy.mean():.2f}")
    print(f"    P-Value: {p_val:.10f} -> {'Significant' if p_val < 0.05 else 'Not Significant'}")
else:
    print("[3] Snow: Not enough snowy days for testing.")
print("-" * 50)

```

Test 4: Temperature (Hot vs Cold - Split by Median)

```

median_temp = df['Temperature'].median()
hot_days = df[df['Temperature'] > median_temp]['Traffic']
cold_days = df[df['Temperature'] <= median_temp]['Traffic']
t_stat, p_val = stats.ttest_ind(hot_days, cold_days, equal_var=False)
print(f"[4] Temperature (Hot vs Cold)")
print(f"    Median Temp: {median_temp:.1f}°C")
print(f"    Mean Traffic (Hot): {hot_days.mean():.2f}")
print(f"    Mean Traffic (Cold): {cold_days.mean():.2f}")
print(f"    P-Value: {p_val:.10f} -> {'Significant' if p_val < 0.05 else 'Not Significant'}")
print("-" * 50)

```

Test 5: Wind (High vs Low Wind - Split by Median)

```

median_wind = df['Wind'].median()
high_wind = df[df['Wind'] > median_wind]['Traffic']
low_wind = df[df['Wind'] <= median_wind]['Traffic']
t_stat, p_val = stats.ttest_ind(high_wind, low_wind, equal_var=False)
print(f"[5] Wind Speed (High vs Low)")
print(f"    P-Value: {p_val:.10f} -> {'Significant' if p_val < 0.05 else 'Not Significant'}")

```

`print("=" * 50)` bundan sonrasında grafiklerle göstermek istiyorum histogramla traffic dist. boxplot kullanarak weekday vs weekend 1 tane cor. Heatmap ve geri kalan her weather condition vs trafik için scatter plot

Answer: Harika bir noktada kalmışsın. Hipotez testleriyle sayısal kanıtı sunduktan sonra, görsel kanıt sunmak analizi çok güçlendirir.

Kodunun `run_part2_analysis` fonksiyonunun içine, hipotez testlerinin hemen altına ekleyebileceğin görselleştirme (Visualization) bloğu aşağıdadır. İstedğin grafik türlerini (Histogram, Boxplot, Heatmap ve Scatter Plotlar) tam olarak kapsayacak şekilde hazırladım.

Koda Eklenecek Bölüm

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
# Set the plot style
```

```
sns.set(style="whitegrid")
```

```
# 1. Histogram for Traffic Distribution
```

```
plt.figure(figsize=(10, 6))
```

```
sns.histplot(df['Traffic'], kde=True, bins=20)
```

```
plt.title('Distribution of Traffic Index')
```

```
plt.xlabel('Traffic Index')
```

```
plt.ylabel('Frequency')
```

```
plt.show()
```

```
# 2. Boxplot for Weekday vs Weekend
```

```
plt.figure(figsize=(10, 6))
```

```
sns.boxplot(x='DayType', y='Traffic', data=df)
```

```
plt.title('Traffic by Day Type')
```

```
plt.show()
```

```
# 3. Correlation Heatmap
```

```
plt.figure(figsize=(12, 8))
```

```
# Select only numeric columns to avoid errors
```

```
numeric_df = df.select_dtypes(include=['float64', 'int64'])
```

```
sns.heatmap(numeric_df.corr(), annot=True, cmap='coolwarm')  
plt.title('Correlation Matrix')  
plt.show()
```

4. Scatter Plots for Weather Variables

```
weather_variables = ['Rain', 'Snow', 'Temperature', 'Wind']
```

```
for var in weather_variables:  
    plt.figure(figsize=(8, 5))  
    sns.scatterplot(x=var, y='Traffic', data=df)  
    plt.title(f'{var} vs Traffic Index')  
    plt.xlabel(var)  
    plt.ylabel('Traffic Index')  
    plt.show()
```

Prompt: veriyi X_train, X_test, y_train, y_test olarak böldüm sırasıyla linear regression ve random forest modeli kur karşılaştırmak için import pandas as pd

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.ensemble import RandomForestRegressor
```

```
from sklearn.metrics import mean_squared_error, r2_score
```

```
def run_ml_pipeline():
```

```
    # Load processed data
```

```
    try:
```

```
        df = pd.read_csv('istanbul_weather_traffic_merged.csv')
```

```
    except FileNotFoundError:
```

```
print("Error: Dataset not found. Please run the previous scripts first.")
```

```
return
```

```
# Preprocessing
```

```
df.rename(columns={  
    'precipitation_sum (mm)': 'Precipitation',  
    'rain_sum (mm)': 'Rain',  
    'snowfall_sum (cm)': 'Snow',  
    'temperature_2m_mean (°C)': 'Temperature',  
    'wind_speed_10m_max (km/h)': 'Wind',  
    'average_traffic_index': 'Traffic'  
}, inplace=True)
```

```
df['date'] = pd.to_datetime(df['date'])
```

```
# Feature Engineering
```

```
df['day_of_week'] = df['date'].dt.dayofweek  
df['is_weekend'] = (df['day_of_week'] >= 5).astype(int)  
df['month'] = df['date'].dt.month
```

```
# Define Feature Matrix (X) and Target Vector (y)
```

```
features = ['Precipitation', 'Rain', 'Snow', 'Temperature', 'Wind', 'day_of_week', 'is_weekend',  
'month']
```

```
target = 'Traffic'
```

```
X = df[features]
```

```
y = df[target]
```

```
# Train-Test Split (Time Series: Shuffle=False)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)
```

```
print("-" * 50)

print(f"Data Split Summary")

print(f"Training Samples: {len(X_train)}")

print(f"Testing Samples: {len(X_test)}")

print("-" * 50)
```

Answer: Bu kodu, gönderdiğin kodun en altına olduğu gibi yapıştırabilirsin:

1. Linear Regression

```
lr = LinearRegression()

lr.fit(X_train, y_train)

predictions_lr = lr.predict(X_test)
```

Calculate RMSE manually if needed

```
rmse_lr = np.sqrt(mean_squared_error(y_test, predictions_lr))

print("Linear Regression RMSE:", rmse_lr)
```

2. Random Forest

```
rf = RandomForestRegressor()

rf.fit(X_train, y_train)

predictions_rf = rf.predict(X_test)
```

```
rmse_rf = np.sqrt(mean_squared_error(y_test, predictions_rf))

print("Random Forest RMSE:", rmse_rf)
```